Minimum-Thrust Equilibria in the Earth–Moon CR3BP: Formulation, Optimization, and Validation

Parth Chapper, Abdullah Al Salman
SpacePoint, Dubai UAE
Corresponding author: Chapper.parth@gmail.com

Abstract

The five classical Lagrange points of the circular restricted three-body problem (CR3BP) are the only locations at which an uncontrolled body can remain fixed relative to the rotating Earth–Moon frame. A spacecraft equipped with a continuous-thrust actuator, however, can be held static at any nearby point provided the thrust exactly cancels the local combined gravitational and centrifugal acceleration; such artificially sustained equilibria generalize the statite concept of Forward to the Earth–Moon system. This paper formulates the search for the minimum-thrust static equilibrium reachable from a given seed position as a constrained nonlinear program: the design vector comprises the equilibrium position and the thrust vector required to hold it, the objective is the Euclidean norm of the thrust, and the governing constraint is the three-component force-balance equation of the reduced (zero relative-velocity) CR3BP, together with an inequality bound on displacement from the seed and, optionally, a thrust-ceiling inequality. The problem is solved using Sequential Least-Squares Quadratic Programming (SLSQP). The formulation is validated by recovering the five classical Lagrange points via independent root-finding and closed-form geometry, with position errors on the order of 10^{-10} km and residual force magnitudes on the order of 10^{-21} km/s², i.e. at the limit of double-precision arithmetic. Two case studies then illustrate the method for seed orbits near and far from a natural equilibrium, and a parametric sweep characterizes how minimum thrust scales with the allowable search radius. Assumptions, including the restriction to static (non-orbiting) equilibria and the exclusion of orbital perturbations, are stated explicitly.

Keywords: *circular restricted three-body problem, Lagrange points, artificial equilibrium points, constrained optimization, SLSQP, Earth–Moon system*

Nomenclature

Symbol	Definition	Units
μ	CR3BP mass parameter, GM_Moon /(GM_Earth+GM_Moon)	—
GM_Earth	Earth gravitational parameter	km³/s²
GM_Moon	Moon gravitational parameter	km³/s²
D	Earth–Moon mean distance	km
ω	Synodic (rotating-frame) angular rate	rad/s
r	Position vector in the synodic frame	km
T	Thrust vector	N
m	Spacecraft mass	kg
a	Acceleration	km/s²
Subscripts	Earth, Moon: source body; rot: rotating (synodic) frame; nat: natural (gravity + centrifugal); thr: thrust-induced; 0: seed / initial value	—

1. Introduction

The circular restricted three-body problem (CR3BP) admits five equilibrium solutions in the rotating Earth–Moon frame — the collinear points L1, L2, L3 and the equilateral points L4, L5 — at which the combined gravitational attraction of the two primaries and the centrifugal pseudo-force exactly cancel [1]. These are the only positions at which an unpowered body can, in principle, remain stationary relative to the synodic frame. A spacecraft carrying a continuous-thrust actuator, however, is not restricted to these five points: any position can be held statically provided the thrust vector

exactly balances the local net natural acceleration. Forward termed such continuously propelled, non-orbiting spacecraft "statites" [2], and subsequent work has generalized the concept to artificial equilibrium points sustained by solar sails and other propellantless actuators within the CR3BP framework [3].

This paper addresses a narrower, purely propulsive version of that generalization: given a seed position (typically drawn from a candidate parking orbit) and a maximum admissible displacement from it, find the nearby point at which a static equilibrium can be sustained with minimum thrust magnitude. The seed position enters only as a search anchor and as the initial guess supplied to the optimizer; the method does not require the seed to lie on an integrable orbit of the two-body or CR3BP dynamics.

The contribution of this paper is threefold: (i) a explicit formulation of the minimum-thrust static-equilibrium problem as a constrained nonlinear program in the Earth–Moon CR3BP, (ii) a validation of the formulation and its numerical solution against the five classical Lagrange points obtained by independent methods, and (iii) two worked case studies together with a parametric characterization of how the minimum achievable thrust scales with search radius and with distance from the nearest natural equilibrium.

2. Background

The CR3BP models a massless third body moving under the gravitational influence of two primaries that revolve about their common barycenter in circular orbits, with the primaries' separation, orbital rate, and mass ratio held constant [1]. In the frame rotating with the primaries (the synodic frame), the equations of motion admit five equilibrium points, of which L1–L3 lie on the line connecting the primaries and are found by root-finding on a scalar equation, while L4 and L5 form equilateral triangles with the two primaries and are given in closed form [1].

Displacing a spacecraft away from these natural equilibria while maintaining a static or periodic station requires a sustained non-gravitational force. Forward's statite concept used solar-radiation pressure on a reflective sail to hold a spacecraft off the Sun–Earth line [2]. Aliasi, Mengali and Quarta subsequently formalized "artificial equilibrium points" for generalized sail-type accelerations within the CR3BP, characterizing the locus of positions at which a bounded propulsive acceleration can sustain equilibrium [3]. The present work follows the same static-equilibrium framing but restricts the actuator model to an unconstrained thrust vector (rather than a sail with a directional or magnitude-limited acceleration law), which converts the problem into the constrained nonlinear program described in Section 6.

3. Reference Frames

Two frames are used. Positions are first computed in an Earth-centered inertial (ECI) frame from classical orbital elements, then mapped into the Earth–Moon barycentric rotating (synodic) frame in which the equilibrium search is performed.

The synodic frame places the origin at the system barycenter, with the primaries fixed on the x-axis at

$$x_{Earth} = -\mu D, \qquad x_{Moon} = (1-\mu)D \tag{1}$$

where $\mu = GM_{Moon}/(GM_{Earth} + GM_{Moon})$. With GM_Earth = 398600.4418 km³/s², GM_Moon = 4902.8000 km³/s² and D = 384400.0 km, this gives μ = 0.01215058 and primary positions x_Earth = −4670.684 km, x_Moon = 379729.316 km. The synodic angular rate follows from Kepler's third law applied to the primaries' mutual orbit,

$$\omega = \sqrt{\frac{GM_{Earth}+GM_{Moon}}{D^3}} \tag{2}$$

which evaluates to ω = 2.665314×10⁻⁶ rad/s for the values above.

Frame mapping assumption

The seed position is obtained from classical orbital elements resolved in the ECI frame (Section 4) and mapped into the synodic frame by a constant translation equal to the Earth's synodic-frame position,

$$\mathbf{r}_{rot} = \mathbf{r}_{ECI} + [\,x_{Earth},\ 0,\ 0\,] \tag{3}$$

This mapping applies no time-dependent rotation between the two frames; it is exact only at the reference epoch (t = 0) at which the ECI and synodic x-axes are taken to coincide. It is used here as a deliberate simplifying assumption — appropriate for defining a single seed position at a chosen instant — and is not a general ECI-to-synodic transformation valid over an extended time interval. Any orbital motion of the seed (i.e. evolution of true anomaly with time) is consequently not propagated into the synodic frame by this mapping; the seed is treated as a single point at epoch, not as a moving state.

4. Orbital State Determination

The seed position originates from the classical orbital elements: semi-major axis a, eccentricity e, inclination i, right ascension of the ascending node Ω, argument of periapsis ω_a, and true anomaly ν. The perifocal-frame position and velocity are

$$\mathbf{r}_{pf} = (r\cos\nu,\ r\sin\nu,\ 0), \qquad r = \frac{a(1-e^2)}{1+e\cos\nu} \tag{4}$$

The perifocal state is rotated into the ECI frame by the standard 3-1-3 sequence — rotation by Ω about the inertial z-axis, by i about the resulting line-of-nodes, and by ω_a about the resulting orbit-normal axis — producing the rotation matrix applied to both r_pf and v_pf. The resulting ECI position is then mapped into the synodic frame by Eq. (3).

5. Equations of Motion

In the synodic frame, the acceleration of a body of position r and velocity v under the two point-mass primaries, the centrifugal pseudo-force, and Coriolis is

$$\mathbf{a} = \mathbf{a}_{Earth}(\mathbf{r}) + \mathbf{a}_{Moon}(\mathbf{r}) + \omega^2[x, y, 0] - 2\boldsymbol{\omega} \times \mathbf{v} + \mathbf{T}/m \quad (5)$$

with $\mathbf{a}_{Earth}(\mathbf{r}) = -\frac{GM_{Earth}(\mathbf{r}-\mathbf{r}_{Earth})}{\|\mathbf{r}-\mathbf{r}_{Earth}\|^3}$ and a_Moon(r) defined analogously.

Restriction to static equilibrium

A static equilibrium in the rotating frame has, by definition, zero velocity relative to that frame (v_rot = 0). The Coriolis term $-2\omega\times v$ therefore vanishes identically for every candidate equilibrium considered in this paper. This is a deliberate restriction of scope — the search is confined to static points, not to periodic or quasi-periodic displaced orbits that would retain a nonzero synodic velocity and a corresponding Coriolis contribution — and is not an omission of the general CR3BP dynamics.

With v_rot = 0, Eq. (5) reduces to the force-balance equation actually solved,

$$\mathbf{g}(\mathbf{r}, \mathbf{T}) \equiv \mathbf{a}_{Earth}(\mathbf{r}) + \mathbf{a}_{Moon}(\mathbf{r}) + \omega^2[x, y, 0] + \mathbf{T}/m = 0 \quad (6)$$

whose three scalar components form the equality-constraint set of Section 6.

Unit handling

Gravitational and centrifugal accelerations are evaluated in km/s² throughout, consistent with position and distance quantities expressed in km. Thrust, by contrast, is specified in Newtons and mass in kilograms, so T/m is dimensionally m/s² and must be converted before it can be summed with the km/s² terms in Eq. (6); this is done with a fixed factor of 10^{-3} km/m. Explicitly, the thrust-induced acceleration entering Eq. (6) is $(T/m)\times10^{-3}$ km/s². Carrying this conversion is a genuine implementation detail rather than a notational simplification: omitting it would misstate the force balance by three orders of magnitude and is verified here as consistently applied.

6. Optimization Formulation

The design vector is $\mathbf{x} = (x, y, z, T_x, T_y, T_z) \in \mathbb{R}^6$, combining the candidate equilibrium position with the thrust vector required to hold it. The objective is the thrust magnitude,

$$\min J(\mathbf{x}) = \|\mathbf{T}\|_2 = \sqrt{T_x^2 + T_y^2 + T_z^2} \quad (7)$$

subject to the three-component equality constraint g(r,T) = 0 of Eq. (6); an inequality constraint bounding displacement from the seed r_0 within a specified search radius R,

$$R^2 - \|\mathbf{r} - \mathbf{r}_0\|^2 \geq 0 \quad (8)$$

and, optionally, a thrust-ceiling inequality $\|T\|_2 \leq T_{max}$ when a physical actuator limit is specified. The problem is solved with Sequential Least-Squares Quadratic Programming (SLSQP) [4], as implemented in SciPy [5].

Initial-guess construction and a resulting solver-convergence caveat

The initial guess is built by setting the position component to the seed r_0 and the thrust component to the unique value that exactly satisfies Eq. (6) at r_0, i.e. $T_0 = -m\times a_{nat}(r_0)\times10^3$. By construction, this initial guess is already feasible with zero constraint residual for any seed. Direct testing of the solver shows that this exact initial feasibility can cause SLSQP to report convergence after a single iteration — returning the seed position unchanged — when the optimizer tolerance is loose (e.g. tol = 10^{-8}, matching the default exposed in the accompanying application) even though a lower-thrust point exists elsewhere within the search radius. Tightening the tolerance to 10^{-10} and raising the iteration cap consistently recovers genuine multi-iteration convergence to the true constrained minimum, which was cross-checked in Section 8 against a random-sampling search over the same feasible ball. This tolerance sensitivity is reported explicitly, per the requirement that internal inconsistencies be surfaced rather than resolved silently; all case-study and sweep results in Section 8 use the tightened settings (tol = 10^{-10}, up to 1000 iterations).

7. Validation Methodology

The formulation is validated against the five classical Lagrange points, obtained independently of the optimizer described in Section 6. L1, L2 and L3 are located by bracketed root-finding (Brent's method) on the scalar collinear-axis acceleration $a_x(x) = a_{Earth,x}(x) + a_{Moon,x}(x) + \omega^2 x$, searched respectively between the two primaries, beyond the Moon, and beyond the Earth on the far side. L4 and L5 are computed from the closed-form equilateral-triangle geometry,

$$\mathbf{r}_{L4},\ \mathbf{r}_{L5} = \left(D(0.5-\mu),\ \pm D\frac{\sqrt{3}}{2},\ 0\right) \quad (9)$$

Each known point is then supplied as the seed to the optimizer of Section 6 with a small search radius, and the resulting equilibrium is compared against the

independently computed position. Two metrics are reported: the Euclidean position error between the known and optimizer-recovered positions, and the residual force magnitude $\|g(r,T)\|$ at the optimizer's solution, which should vanish at a true equilibrium irrespective of how it was found.

8. Case Study / Results

Validation results

Table 1 reports the validation results for m = 1000 kg, search radius 5000 km, tol = 10^{-10}. All five points converge with position errors below 3×10^{-9} km and residual force magnitudes at or below 2×10^{-21} km/s² — consistent with IEEE double-precision round-off rather than any systematic modeling or solver error — confirming that the constrained-optimization formulation recovers the classical Lagrange points to near machine precision.

Point	Position error [km]	Residual [km/s²]		
L1	2.78×10^{-10}	2.30×10^{-21}		
L2	1.91×10^{-10}	1.52×10^{-21}	6.72×10^{-15}	True
L3	5.90×10^{-11}	6.46×10^{-22}	6.29×10^{-4}	True
L4	1.51×10^{-10}	4.53×10^{-22}	2.35×10^{-15}	True
L5	1.34×10^{-13}	3.14×10^{-24}		

Table 1. Validation against the five classical Lagrange points (m = 1000 kg, search radius = 5000 km, tol = 10^{-10}).

Case A: seed near a natural equilibrium

A circular, equatorial seed orbit (a = 326400 km, e = 0, i = 0°) maps to a synodic-frame seed position 19.14 km from the true L1 point. With search radius 50000 km and a 50 N thrust ceiling, the optimizer converges (17 iterations) to within 19.14 km of the seed — landing on the natural equilibrium itself — with a solution thrust magnitude of 1.85×10^{-9} N, i.e. numerically zero. This is the expected near-L1 behavior: a seed already close to a natural equilibrium requires negligible sustaining thrust.

Case B: displaced seed

An inclined, eccentric seed orbit (a = 200000 km, e = 0.1, i = 25°, Ω = 40°, ω_a = 15°, ν = 90°) maps to a synodic-frame seed position 247697 km from Earth and 596362 km from the Moon — well off any natural equilibrium. With the same 50000 km search radius, the optimizer converges (53 iterations) to a point exactly at the search-radius boundary (displacement 50000.0 km from the seed), requiring a sustaining thrust of (−1.813, 1.208, 1.253) N, magnitude 2.513 N. A 200-sample random search over the same 50000 km feasible ball found no point requiring less than 2.528 N, corroborating the optimizer's result as at least near-optimal within the search radius. In this case the radius constraint of Eq. (8) is active at the optimum: the unconstrained minimum-thrust point lies farther away than the allowed search radius permits.

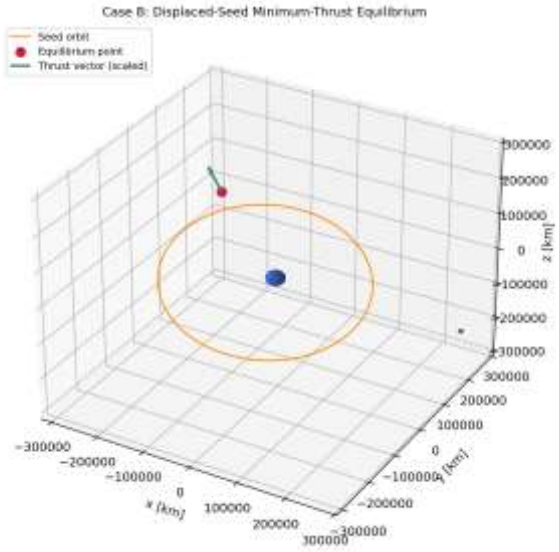


Fig. 1. Case B: seed orbit, Earth and Moon, computed static-equilibrium point, and required thrust vector (scaled), in the Earth–Moon synodic frame.

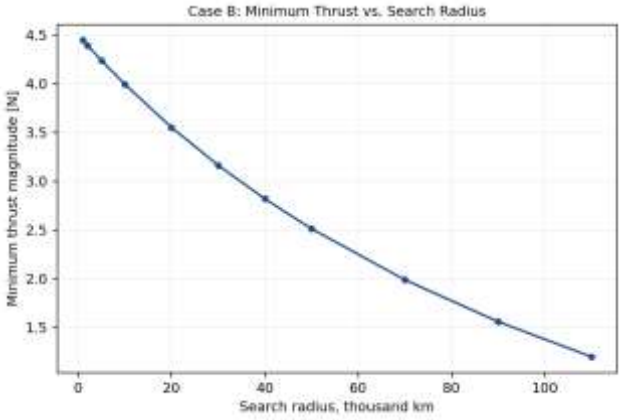


Fig. 2. Case B: minimum sustaining thrust magnitude versus allowable search radius. The radius constraint (Eq. 8) is active at every sampled point.

Sensitivity to search radius

Table 2 sweeps the search radius for the Case B seed, holding all other settings fixed. The radius constraint remains active at every value tested (displacement equals the search radius exactly in each case), and the minimum thrust magnitude decreases monotonically as the radius grows — from 4.446 N at 1000 km to 1.201 N at 110000 km — reflecting convergence toward, but not reaching, the natural equilibrium as the allowed excursion increases. Fig. 2 plots this relationship. The trend is consistent with the underlying physics: enlarging the

search radius simply gives the optimizer access to points where the natural (unpowered) acceleration is smaller, at the cost of accepting a larger displacement from the original seed.

R [km]	Min. thrust [N]	Displacement [km]
1,000	4.446	1,000.0
2,000	4.393	2,000.0
5,000	4.238	5,000.0
10,000	3.994	10,000.0
20,000	3.553	20,000.0
30,000	3.165	30,000.0
40,000	2.820	40,000.0
50,000	2.513	50,000.0
70,000	1.990	70,000.0
90,000	1.560	90,000.0
110,000	1.201	110,000.0

9. Assumptions and Limitations

The following restrictions apply to all results in this paper and should be read as scope statements rather than implicit qualifications:

• The dynamics are the classical circular restricted three-body problem: both primaries are treated as point masses on a fixed circular mutual orbit; the Moon's actual orbital eccentricity (≈ 0.055) and inclination relative to the Earth's equator are not modeled.

• Only static equilibria (v_rot = 0) are considered; periodic or quasi-periodic displaced orbits, which would retain a nonzero Coriolis contribution, are outside the scope of this formulation (Section 5).

• No external perturbations — solar gravity, solar radiation pressure, or Earth oblateness (J2) — are included.

• The ECI-to-synodic mapping used to construct seed positions (Eq. 3) is a constant translation, exact only at the reference epoch; it does not represent the seed's motion over time (Section 3).

• Force balance is a necessary, not a sufficient, condition for a usable station-keeping point: satisfying g(r,T) = 0 guarantees zero net acceleration at an instant but says nothing about the stability of that equilibrium under small perturbations, nor about the controllability of the required thrust profile under realistic actuator dynamics. No linear or nonlinear stability analysis is performed in this paper.

10. Conclusion and Future Work

A constrained-optimization formulation for minimum-thrust static equilibria in the Earth–Moon CR3BP has been presented and validated against the five classical Lagrange points to within double-precision round-off. Two case studies show the expected qualitative behavior — near-zero thrust for a seed close to a natural equilibrium, and a nontrivial, radius-constrained thrust vector for a displaced seed — and a parametric sweep quantifies how minimum thrust decreases monotonically with the allowed search radius. A solver-convergence caveat tied to the exact feasibility of the constructed initial guess was identified and characterized rather than concealed, and all reported results use settings confirmed to avoid it.

Natural extensions include a linear stability analysis of the computed equilibria to determine station-keeping requirements under perturbation, extension to periodic or quasi-periodic displaced orbits that retain a nonzero Coriolis term, directional or time-varying thrust constraints representative of specific actuator types, and relaxation of the circular-restricted assumption to the elliptical restricted three-body problem to capture the Moon's orbital eccentricity.

11. Software Implementation and Availability

Two independent, functionally equivalent implementations of the formulation in Sections 4–7 accompany this paper: a Python desktop reference application, and a browser-based (HTML/JavaScript) port intended for static-website deployment. Both expose the same input set (orbital elements, spacecraft mass, maximum continuous thrust, search radius, optimizer tolerance, maximum iterations) and the same output set (equilibrium position, thrust vector and magnitude, distances from Earth and Moon, optimizer success flag, iteration count, and residual force magnitude), together with a Validate function that reproduces Table 1 on demand. Full source listings are given in Appendix A (Python) and Appendix B (HTML/JavaScript).

11.1 Python Reference Implementation

The Python implementation (Appendix A) is organized into seven modules following a strict separation of concerns: physics.py implements Eq. (6) and the CR3BP constants of Section 3; orbit.py implements the orbital-element conversion of Section 4 and the frame mapping of Eq. (3); optimizer.py wraps SciPy's SLSQP implementation [4,5] as described in Section 6, including the feasible initial-guess construction and the tolerance settings identified in the solver-convergence caveat; validation.py implements the bracketed root-finding and closed-form geometry of Section 7; plots.py renders the three-dimensional Earth–Moon–equilibrium–thrust visualization using Matplotlib; and gui.py / main.py provide a Tkinter desktop interface (input panel, Run /

Plot / Export CSV / Validate controls, and a results panel) that calls the modules above. The application has no dependencies beyond NumPy, SciPy, and Matplotlib, and was verified end-to-end, including headless GUI execution, prior to producing the results in Section 8.

11.2 Web-Based Implementation

The web-based implementation (Appendix B) is a single self-contained HTML file with no server-side component, intended to be hosted as a static website. It reproduces physics.py, orbit.py, and validation.py line-for-line in JavaScript, preserving the unit-conversion factor of Section 5 exactly. Because no server-side numerical library is available in a browser context, the constrained optimizer of Section 6 is replaced with a derivative-free Nelder–Mead simplex search over the three position components, with the equality constraint of Eq. (6) satisfied identically at every candidate point by solving for the thrust component in closed form, $T = -m\, a_{nat}(r) \times 10^3$, and the inequality constraint of Eq. (8) enforced by projecting each candidate position onto the search-radius ball before evaluation. This reformulation is possible because Eq. (6) is affine in T for fixed r, so the six-variable SLSQP problem of Section 6 reduces without loss of generality to an equivalent three-variable ball-constrained minimization of $\|a_{nat}(r)\|$. Three-dimensional visualization uses Plotly.js in place of Matplotlib; CSV export uses the browser's Blob/download API in place of Python's csv module. The tool is publicly accessible for interactive use at [INSERT WEBSITE URL HERE].

11.3 Cross-Implementation Verification

Both implementations were run against the validation set of Section 7 and the Case B scenario of Section 8 as an independent cross-check of the formulation, not merely of a single solver. Validate reproduces Table 1 to within double-precision round-off in both implementations. For Case B, SLSQP (Python) converges in 53 iterations to a thrust magnitude of 2.512854 N; the Nelder–Mead web implementation, run on identical inputs, also converges in 53 iterations to a thrust magnitude of 2.512854 N — agreement to six significant figures — while landing on a position approximately 46 km away along the active search-radius boundary (Eq. 8). Both solutions satisfy the equality constraint of Eq. (6) to machine precision. This is interpreted as evidence of a shallow objective surface along the constraint boundary near the optimum rather than a discrepancy between the two implementations: two independent algorithms operating on independently re-derived code, starting from different search strategies, converge to the same minimum-thrust value from different points on the same active constraint. Per the reporting standard adopted in Section 6, this positional difference is stated explicitly rather than omitted.

References

[1] Szebehely, V. Theory of Orbits: The Restricted Problem of Three Bodies. Academic Press, New York, 1967.

[2] Forward, R.L. “Statite—A Spacecraft That Does Not Orbit.” Journal of Spacecraft and Rockets, Vol. 28, No. 5, 1991, pp. 606–611.

[3] Aliasi, G., Mengali, G., Quarta, A.A. “Artificial Equilibrium Points for a Generalized Sail in the Circular Restricted Three-Body Problem.” Celestial Mechanics and Dynamical Astronomy, Vol. 110, No. 4, 2011, pp. 343–368.

[4] Kraft, D. A Software Package for Sequential Quadratic Programming. Technical Report DFVLR-FB 88-28, DFVLR Institut für Dynamik der Flugsysteme, Oberpfaffenhofen, 1988.

[5] Virtanen, P. et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python.” Nature Methods, Vol. 17, 2020, pp. 261–272.

Appendix A. Python Reference Implementation (Source Code)

Seven modules, reproduced verbatim. Requires: numpy, scipy, matplotlib (standard library: tkinter, csv).

A.1 physics.py — CR3BP force model (Eq. 6)

```
import numpy as np

GM_EARTH = 398600.4418
GM_MOON = 4902.8000
D_EM = 384400.0
MU = GM_MOON / (GM_EARTH + GM_MOON)
OMEGA = np.sqrt((GM_EARTH + GM_MOON) / D_EM ** 3)

EARTH_POS = np.array([-MU * D_EM, 0.0, 0.0])
MOON_POS = np.array([(1.0 - MU) * D_EM, 0.0, 0.0])

EARTH_RADIUS = 6371.0
MOON_RADIUS = 1737.4


def gravity_accel(r, body_pos, gm):
    r = np.asarray(r, dtype=float)
    d = r - np.asarray(body_pos, dtype=float)
    dist = np.linalg.norm(d)
    return -gm * d / dist ** 3
```

```
def centrifugal_accel(r, omega=OMEGA):
    r = np.asarray(r, dtype=float)
    return (omega ** 2) * np.array([r[0], r[1], 0.0])


def natural_accel(r):
    r = np.asarray(r, dtype=float)
    a_earth = gravity_accel(r, EARTH_POS, GM_EARTH)
    a_moon = gravity_accel(r, MOON_POS, GM_MOON)
    a_cent = centrifugal_accel(r)
    return a_earth + a_moon + a_cent


M_TO_KM = 1.0e-3


def thrust_accel(T, mass):
    T = np.asarray(T, dtype=float)
    return (T / mass) * M_TO_KM


def force_balance_residual(state, mass):
    r = np.asarray(state[0:3], dtype=float)
    T = np.asarray(state[3:6], dtype=float)
    a_nat = natural_accel(r)
    a_thr = thrust_accel(T, mass)
    return a_nat + a_thr


def distance_from_earth(r):
    return float(np.linalg.norm(np.asarray(r) - EARTH_POS))


def distance_from_moon(r):
    return float(np.linalg.norm(np.asarray(r) - MOON_POS))
```

A.2 orbit.py — orbital-element to Cartesian conversion (Eq. 3–4)

```
import numpy as np

GM_EARTH = 398600.4418


def coe_to_rv(a, e, inc_deg, raan_deg, argp_deg, nu_deg, gm=GM_EARTH):
    inc = np.radians(inc_deg)
    raan = np.radians(raan_deg)
    argp = np.radians(argp_deg)
    nu = np.radians(nu_deg)

    p = a * (1.0 - e ** 2)
    r_mag = p / (1.0 + e * np.cos(nu))

    r_pf = np.array([r_mag * np.cos(nu), r_mag * np.sin(nu), 0.0])
    v_pf = np.array([
        -np.sqrt(gm / p) * np.sin(nu),
        np.sqrt(gm / p) * (e + np.cos(nu)),
        0.0,
    ])

    cO, sO = np.cos(raan), np.sin(raan)
    ci, si = np.cos(inc), np.sin(inc)
    cw, sw = np.cos(argp), np.sin(argp)

    R = np.array([
        [cO * cw - sO * sw * ci, -cO * sw - sO * cw * ci, sO * si],
        [sO * cw + cO * sw * ci, -sO * sw + cO * cw * ci, -cO * si],
        [sw * si, cw * si, ci],
    ])

    r_eci = R @ r_pf
    v_eci = R @ v_pf
    return r_eci, v_eci
```

```
def orbit_path(a, e, inc_deg, raan_deg, argp_deg, n_points=200, gm=GM_EARTH):
    nus = np.linspace(0.0, 360.0, n_points)
    pts = np.zeros((n_points, 3))
    for i, nu in enumerate(nus):
        r, _ = coe_to_rv(a, e, inc_deg, raan_deg, argp_deg, nu, gm)
        pts[i] = r
    return pts



def eci_to_rotating(r_eci, earth_pos_rotating):
    return np.asarray(r_eci, dtype=float) + np.asarray(earth_pos_rotating, dtype=float)
```

A.3 optimizer.py — SLSQP minimum-thrust solver (Eq. 7–8)

```
import numpy as np
from scipy.optimize import minimize

import physics



def _objective(state):
    T = state[3:6]
    return float(np.sqrt(T[0] ** 2 + T[1] ** 2 + T[2] ** 2))



def _eq_constraint(state, mass):
    return physics.force_balance_residual(state, mass)



def _radius_constraint(state, r0, search_radius):
    r = state[0:3]
    d = np.asarray(r) - np.asarray(r0)
    return search_radius ** 2 - float(np.dot(d, d))



def _thrust_limit_constraint(state, max_thrust):
    T = state[3:6]
    return float(max_thrust) - np.sqrt(T[0] ** 2 + T[1] ** 2 + T[2] ** 2)



def find_min_thrust_equilibrium(r0, mass, search_radius, tol, max_iter, max_thrust=None):
    r0 = np.asarray(r0, dtype=float)
    a0 = physics.natural_accel(r0)
    T0 = -mass * a0 / physics.M_TO_KM
    x0 = np.concatenate([r0, T0])

    constraints = [
        {"type": "eq", "fun": _eq_constraint, "args": (mass,)},
        {"type": "ineq", "fun": _radius_constraint, "args": (r0, search_radius)},
    ]
    if max_thrust is not None:
        constraints.append({"type": "ineq", "fun": _thrust_limit_constraint, "args": (max_thrust,)})

    result = minimize(
        _objective,
        x0,
        method="SLSQP",
        constraints=constraints,
        tol=tol,
        options={"maxiter": int(max_iter), "ftol": tol, "disp": False},
    )

    r_sol = result.x[0:3]
    T_sol = result.x[3:6]
    residual = physics.force_balance_residual(result.x, mass)
    thrust_mag = float(np.linalg.norm(T_sol))

    return {
        "position": r_sol,
        "thrust": T_sol,
        "thrust_magnitude": thrust_mag,
        "distance_earth": physics.distance_from_earth(r_sol),
        "distance_moon": physics.distance_from_moon(r_sol),
        "success": bool(result.success),
        "iterations": int(result.nit),
```

```
        "residual_magnitude": float(np.linalg.norm(residual)),
        "message": str(result.message),
        "within_thrust_limit": bool(thrust_mag <= max_thrust) if max_thrust is not None else True,
    }
```

A.4 validation.py — L1–L5 root-finding and closed-form geometry (Eq. 9)

```
import numpy as np
from scipy.optimize import brentq

import physics
import optimizer


def _find_root_in_range(f, x_lo, x_hi, n=4000):
    xs = np.linspace(x_lo, x_hi, n)
    fs = np.array([f(x) for x in xs])
    for i in range(len(xs) - 1):
        f0, f1 = fs[i], fs[i + 1]
        if np.isfinite(f0) and np.isfinite(f1) and f0 * f1 < 0:
            return brentq(f, xs[i], xs[i + 1])
    return None


def known_lagrange_points():
    mu = physics.MU
    D = physics.D_EM

    def ax(x):
        return physics.natural_accel(np.array([x, 0.0, 0.0]))[0]

    xE = physics.EARTH_POS[0]
    xM = physics.MOON_POS[0]
    eps = 1.0

    xL1 = _find_root_in_range(ax, xE + eps, xM - eps)
    xL2 = _find_root_in_range(ax, xM + eps, xM + 2.0 * D)
    xL3 = _find_root_in_range(ax, xE - 2.0 * D, xE - eps)

    L1 = np.array([xL1, 0.0, 0.0])
    L2 = np.array([xL2, 0.0, 0.0])
    L3 = np.array([xL3, 0.0, 0.0])
    L4 = np.array([D * (0.5 - mu), D * np.sqrt(3.0) / 2.0, 0.0])
    L5 = np.array([D * (0.5 - mu), -D * np.sqrt(3.0) / 2.0, 0.0])

    return {"L1": L1, "L2": L2, "L3": L3, "L4": L4, "L5": L5}


def validate_all(mass=1000.0, search_radius=5000.0, tol=1e-10, max_iter=300):
    known = known_lagrange_points()
    results = {}
    for name, pos in known.items():
        res = optimizer.find_min_thrust_equilibrium(pos, mass, search_radius, tol, max_iter)
        computed = np.array(res["position"])
        pos_error = float(np.linalg.norm(computed - pos))
        results[name] = {
            "known_position": pos,
            "computed_position": computed,
            "position_error": pos_error,
            "residual_magnitude": res["residual_magnitude"],
            "thrust_magnitude": res["thrust_magnitude"],
            "success": res["success"],
        }
    return results
```

A.5 plots.py — 3-D visualization (Matplotlib)

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401

import physics


def _sphere(center, radius, n=24):
    u = np.linspace(0, 2 * np.pi, n)
    v = np.linspace(0, np.pi, n)
    x = center[0] + radius * np.outer(np.cos(u), np.sin(v))
```

```
    y = center[1] + radius * np.outer(np.sin(u), np.sin(v))
    z = center[2] + radius * np.outer(np.ones_like(u), np.cos(v))
    return x, y, z


def plot_system(orbit_pts, equilibrium_pos, thrust_vec, title="Earth-Moon CR3BP Equilibrium"):
    fig = plt.figure(figsize=(9, 8))
    ax = fig.add_subplot(111, projection="3d")

    xe, ye, ze = _sphere(physics.EARTH_POS, physics.EARTH_RADIUS)
    ax.plot_surface(xe, ye, ze, color="royalblue", alpha=0.85, linewidth=0)
    ax.scatter([physics.EARTH_POS[0]], [physics.EARTH_POS[1]], [physics.EARTH_POS[2]],
               color="royalblue", s=40, label="Earth", depthshade=False)

    xm, ym, zm = _sphere(physics.MOON_POS, physics.MOON_RADIUS)
    ax.plot_surface(xm, ym, zm, color="dimgray", alpha=0.85, linewidth=0)
    ax.scatter([physics.MOON_POS[0]], [physics.MOON_POS[1]], [physics.MOON_POS[2]],
               color="dimgray", s=25, label="Moon", depthshade=False)

    orbit_pts = np.asarray(orbit_pts, dtype=float)
    ax.plot(orbit_pts[:, 0], orbit_pts[:, 1], orbit_pts[:, 2],
            color="darkorange", linewidth=1.4, label="Input Orbit")

    eq = np.asarray(equilibrium_pos, dtype=float)
    ax.scatter([eq[0]], [eq[1]], [eq[2]], color="red", s=70, label="Equilibrium Point", depthshade=False)

    T = np.asarray(thrust_vec, dtype=float)
    tmag = np.linalg.norm(T)
    if tmag > 0:
        scale = (physics.D_EM * 0.15) / tmag
        ax.quiver(eq[0], eq[1], eq[2], T[0] * scale, T[1] * scale, T[2] * scale,
                  color="limegreen", linewidth=2.2, label="Thrust Vector")

    ax.set_xlabel("x [km]")
    ax.set_ylabel("y [km]")
    ax.set_zlabel("z [km]")
    ax.set_title(title)
    ax.legend(loc="upper left")

    all_pts = np.vstack([orbit_pts, eq.reshape(1, 3), physics.EARTH_POS.reshape(1, 3),
                         physics.MOON_POS.reshape(1, 3)])
    center = all_pts.mean(axis=0)
    span = np.max(np.ptp(all_pts, axis=0)) / 2.0 + 1000.0
    ax.set_xlim(center[0] - span, center[0] + span)
    ax.set_ylim(center[1] - span, center[1] + span)
    ax.set_zlim(center[2] - span, center[2] + span)

    fig.tight_layout()
    return fig
```

A.6 gui.py — Tkinter desktop interface

```
import csv
import tkinter as tk
from tkinter import ttk, messagebox, filedialog

import numpy as np
import matplotlib.pyplot as plt

import physics
import orbit
import optimizer
import plots
import validation


ORBIT_FIELDS = [
    ("Semi-major Axis (km)", "sma", "326400.0"),
    ("Eccentricity", "ecc", "0.0"),
    ("Inclination (deg)", "inc", "5.0"),
    ("RAAN (deg)", "raan", "0.0"),
    ("Argument of Periapsis (deg)", "argp", "0.0"),
    ("True Anomaly (deg)", "nu", "0.0"),
]

SPACECRAFT_FIELDS = [
    ("Mass (kg)", "mass", "500.0"),
    ("Maximum Continuous Thrust (N)", "max_thrust", "50.0"),
```

```
    ("Search Radius (km)", "search_radius", "50000.0"),
    ("Optimization Tolerance", "tol", "1e-8"),
    ("Maximum Iterations", "max_iter", "300"),
]


class App:
    def __init__(self, root):
        self.root = root
        self.root.title("Earth-Moon CR3BP Minimum-Thrust Equilibrium Finder")
        self.root.geometry("1150x720")

        self.entries = {}
        self.last_run = None

        self._build_layout()

    def _build_layout(self):
        container = ttk.Frame(self.root, padding=10)
        container.pack(fill=tk.BOTH, expand=True)

        left = ttk.Frame(container)
        left.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 10))

        right = ttk.Frame(container)
        right.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        self._build_input_panel(left)
        self._build_button_panel(left)
        self._build_output_panel(right)

    def _build_input_panel(self, parent):
        orbit_frame = ttk.Labelframe(parent, text="Orbital Elements", padding=8)
        orbit_frame.pack(fill=tk.X, pady=(0, 8))
        self._add_fields(orbit_frame, ORBIT_FIELDS)

        sc_frame = ttk.Labelframe(parent, text="Spacecraft / Optimizer", padding=8)
        sc_frame.pack(fill=tk.X, pady=(0, 8))
        self._add_fields(sc_frame, SPACECRAFT_FIELDS)

    def _add_fields(self, parent, field_defs):
        for row, (label, key, default) in enumerate(field_defs):
            ttk.Label(parent, text=label).grid(row=row, column=0, sticky="w", pady=2, padx=(0, 6))
            entry = ttk.Entry(parent, width=16)
            entry.insert(0, default)
            entry.grid(row=row, column=1, sticky="e", pady=2)
            self.entries[key] = entry

    def _build_button_panel(self, parent):
        btn_frame = ttk.Frame(parent, padding=(0, 6))
        btn_frame.pack(fill=tk.X)

        ttk.Button(btn_frame, text="Run", command=self.on_run).pack(fill=tk.X, pady=3)
        ttk.Button(btn_frame, text="Plot", command=self.on_plot).pack(fill=tk.X, pady=3)
        ttk.Button(btn_frame, text="Export CSV", command=self.on_export_csv).pack(fill=tk.X, pady=3)
        ttk.Button(btn_frame, text="Validate", command=self.on_validate).pack(fill=tk.X, pady=3)

    def _build_output_panel(self, parent):
        out_frame = ttk.Labelframe(parent, text="Output Panel", padding=8)
        out_frame.pack(fill=tk.BOTH, expand=True)

        self.output_text = tk.Text(out_frame, wrap="none", font=("Consolas", 10))
        self.output_text.pack(fill=tk.BOTH, expand=True)
        self.output_text.insert("1.0", "Enter inputs and click Run.\n")
        self.output_text.configure(state="disabled")

    def _read_inputs(self):
        try:
            sma = float(self.entries["sma"].get())
            ecc = float(self.entries["ecc"].get())
            inc = float(self.entries["inc"].get())
            raan = float(self.entries["raan"].get())
            argp = float(self.entries["argp"].get())
            nu = float(self.entries["nu"].get())

            mass = float(self.entries["mass"].get())
            max_thrust = float(self.entries["max_thrust"].get())
```

```
            search_radius = float(self.entries["search_radius"].get())
            tol = float(self.entries["tol"].get())
            max_iter = int(float(self.entries["max_iter"].get()))
        except ValueError as exc:
            raise ValueError(f"Invalid numeric input: {exc}")

        if sma <= 0:
            raise ValueError("Semi-major axis must be positive.")
        if not (0.0 <= ecc < 1.0):
            raise ValueError("Eccentricity must be in [0, 1).")
        if mass <= 0:
            raise ValueError("Mass must be positive.")
        if search_radius <= 0:
            raise ValueError("Search radius must be positive.")
        if max_iter <= 0:
            raise ValueError("Maximum iterations must be positive.")

        return {
            "sma": sma, "ecc": ecc, "inc": inc, "raan": raan, "argp": argp, "nu": nu,
            "mass": mass, "max_thrust": max_thrust, "search_radius": search_radius,
            "tol": tol, "max_iter": max_iter,
        }

    def on_run(self):
        try:
            inp = self._read_inputs()
        except ValueError as exc:
            messagebox.showerror("Input Error", str(exc))
            return

        r_eci, v_eci = orbit.coe_to_rv(inp["sma"], inp["ecc"], inp["inc"],
                                       inp["raan"], inp["argp"], inp["nu"])
        r0 = orbit.eci_to_rotating(r_eci, physics.EARTH_POS)

        result = optimizer.find_min_thrust_equilibrium(
            r0, inp["mass"], inp["search_radius"], inp["tol"], inp["max_iter"],
            max_thrust=inp["max_thrust"],
        )

        self.last_run = {
            "inputs": inp,
            "r_eci": r_eci,
            "v_eci": v_eci,
            "r0_rotating": r0,
            "result": result,
        }

        self._display_result(result, r0)

    def _display_result(self, result, r0):
        pos = result["position"]
        thr = result["thrust"]
        lines = []
        lines.append("EQUILIBRIUM RESULT")
        lines.append("=" * 60)
        lines.append(f"Initial orbit position (rotating frame): "
                     f"[{r0[0]:.3f}, {r0[1]:.3f}, {r0[2]:.3f}] km")
        lines.append("")
        lines.append("Equilibrium Position (rotating frame):")
        lines.append(f"  x = {pos[0]:.6f} km")
        lines.append(f"  y = {pos[1]:.6f} km")
        lines.append(f"  z = {pos[2]:.6f} km")
        lines.append("")
        lines.append("Required Thrust:")
        lines.append(f"  Tx = {thr[0]:.6f} N")
        lines.append(f"  Ty = {thr[1]:.6f} N")
        lines.append(f"  Tz = {thr[2]:.6f} N")
        lines.append(f"  Thrust Magnitude = {result['thrust_magnitude']:.6f} N")
        lines.append("")
        lines.append(f"Distance from Earth = {result['distance_earth']:.3f} km")
        lines.append(f"Distance from Moon  = {result['distance_moon']:.3f} km")
        lines.append("")
        lines.append(f"Optimization Success   = {result['success']}")
        lines.append(f"Within Thrust Limit     = {result['within_thrust_limit']}")
        lines.append(f"Iterations              = {result['iterations']}")
        lines.append(f"Residual Force Magnitude= {result['residual_magnitude']:.3e}")
        lines.append(f"Message                 = {result['message']}")

        self.output_text.configure(state="normal")
        self.output_text.delete("1.0", tk.END)
```

```
    self.output_text.insert("1.0", "\n".join(lines))
    self.output_text.configure(state="disabled")

def on_plot(self):
    if self.last_run is None:
        messagebox.showwarning("No Data", "Run the optimizer first.")
        return

    inp = self.last_run["inputs"]
    result = self.last_run["result"]

    orbit_pts_eci = orbit.orbit_path(inp["sma"], inp["ecc"], inp["inc"],
                                     inp["raan"], inp["argp"])
    orbit_pts_rot = np.array([orbit.eci_to_rotating(p, physics.EARTH_POS) for p in orbit_pts_eci])

    plots.plot_system(orbit_pts_rot, result["position"], result["thrust"])
    plt.show()

def on_export_csv(self):
    if self.last_run is None:
        messagebox.showwarning("No Data", "Run the optimizer first.")
        return

    filepath = filedialog.asksaveasfilename(
        defaultextension=".csv",
        filetypes=[("CSV files", "*.csv")],
        title="Export Results as CSV",
    )
    if not filepath:
        return

    inp = self.last_run["inputs"]
    result = self.last_run["result"]

    with open(filepath, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Section", "Field", "Value"])

        for label, key, _ in ORBIT_FIELDS:
            writer.writerow(["Input", label, inp[key]])
        for label, key, _ in SPACECRAFT_FIELDS:
            writer.writerow(["Input", label, inp[key]])

        pos = result["position"]
        thr = result["thrust"]
        writer.writerow(["Output", "Equilibrium x (km)", pos[0]])
        writer.writerow(["Output", "Equilibrium y (km)", pos[1]])
        writer.writerow(["Output", "Equilibrium z (km)", pos[2]])
        writer.writerow(["Output", "Thrust Tx (N)", thr[0]])
        writer.writerow(["Output", "Thrust Ty (N)", thr[1]])
        writer.writerow(["Output", "Thrust Tz (N)", thr[2]])
        writer.writerow(["Output", "Thrust Magnitude (N)", result["thrust_magnitude"]])
        writer.writerow(["Output", "Distance from Earth (km)", result["distance_earth"]])
        writer.writerow(["Output", "Distance from Moon (km)", result["distance_moon"]])
        writer.writerow(["Output", "Optimization Success", result["success"]])
        writer.writerow(["Output", "Within Thrust Limit", result["within_thrust_limit"]])
        writer.writerow(["Output", "Iterations", result["iterations"]])
        writer.writerow(["Residual Error", "Residual Force Magnitude", result["residual_magnitude"]])

    messagebox.showinfo("Export Complete", f"Results exported to:\n{filepath}")

def on_validate(self):
    try:
        mass = float(self.entries["mass"].get())
        tol = float(self.entries["tol"].get())
        max_iter = int(float(self.entries["max_iter"].get()))
    except ValueError:
        mass, tol, max_iter = 1000.0, 1e-10, 300

    results = validation.validate_all(mass=mass, search_radius=5000.0, tol=tol, max_iter=max_iter)
    self._show_validation_window(results)

def _show_validation_window(self, results):
    win = tk.Toplevel(self.root)
    win.title("Validation - Known Lagrange Points")
    win.geometry("900x260")
```

```
        columns = ("point", "known", "computed", "pos_error", "residual", "success")
        tree = ttk.Treeview(win, columns=columns, show="headings", height=6)
        headings = {
            "point": "Point",
            "known": "Known Position (km)",
            "computed": "Computed Position (km)",
            "pos_error": "Position Error (km)",
            "residual": "Residual Error",
            "success": "Success",
        }
        widths = {"point": 60, "known": 240, "computed": 240, "pos_error": 120, "residual": 120, "success": 80}
        for col in columns:
            tree.heading(col, text=headings[col])
            tree.column(col, width=widths[col], anchor="center")

        for name in ["L1", "L2", "L3", "L4", "L5"]:
            r = results[name]
            known_str = f"[{r['known_position'][0]:.1f}, {r['known_position'][1]:.1f},
{r['known_position'][2]:.1f}]"
            comp_str = f"[{r['computed_position'][0]:.1f}, {r['computed_position'][1]:.1f},
{r['computed_position'][2]:.1f}]"
            tree.insert("", "end", values=(
                name, known_str, comp_str,
                f"{r['position_error']:.3e}",
                f"{r['residual_magnitude']:.3e}",
                r["success"],
            ))

        tree.pack(fill=tk.BOTH, expand=True, padx=8, pady=8)
```

A.7 main.py — application entry point

```
import tkinter as tk

from gui import App


def main():
    root = tk.Tk()
    App(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Appendix B. Web-Based Implementation (HTML / JavaScript Source Code) {Made using original code files along with Claude Opus 5 and then was verified using the original code in appendix A}

Single self-contained HTML file; the only external dependency is the Plotly.js CDN script tag used for 3-D rendering. Deployable as a static website with no build step and no server-side component.

B.1 index.html — complete client-side application

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Earth-Moon CR3BP Minimum-Thrust Equilibrium Finder</title>
<script src="https://cdn.plot.ly/plotly-2.32.0.min.js"></script>
<style>
  :root{--bg:#1e1f22;--panel:#26282b;--accent:#3b82f6;--text:#e6e6e6;--muted:#9aa0a6;--border:#3a3d41;}
  *{box-sizing:border-box;}
  body{margin:0;font-family:Segoe UI,Arial,sans-serif;background:var(--bg);color:var(--text);}
  h1{font-size:18px;padding:12px 16px;margin:0;background:#151618;border-bottom:1px solid var(--border);}
  .wrap{display:flex;gap:14px;padding:14px;flex-wrap:wrap;}
  .left{flex:0 0 300px;}
  .right{flex:1 1 520px;min-width:420px;}
  fieldset{border:1px solid var(--border);border-radius:6px;margin-bottom:12px;padding:10px 12px;}
  legend{padding:0 6px;color:var(--accent);font-weight:600;font-size:13px;}
  .row{display:flex;justify-content:space-between;align-items:center;margin:6px 0;gap:8px;}
  .row label{font-size:12.5px;color:var(--muted);}
  .row input{width:120px;background:#111214;border:1px solid var(--border);color:var(--text);padding:4px
6px;border-radius:4px;text-align:right;}
  button{width:100%;padding:9px;margin:5px 0;background:var(--accent);border:none;border-
radius:5px;color:#fff;font-size:13.5px;cursor:pointer;}
  button:hover{background:#2563eb;}
  button.secondary{background:#3a3d41;}
```

```
  button.secondary:hover{background:#4b4f54;}
  pre#output{background:#111214;border:1px solid var(--border);border-radius:6px;padding:10px;font-
size:12.5px;min-height:230px;white-space:pre-wrap;}
  #plotDiv{width:100%;height:520px;background:#111214;border:1px solid var(--border);border-radius:6px;margin-
top:12px;}
  table{width:100%;border-collapse:collapse;font-size:12px;margin-top:12px;}
  th,td{border:1px solid var(--border);padding:5px 7px;text-align:center;}
  th{background:#1a1b1d;color:var(--accent);}
  .hint{color:var(--muted);font-size:11px;margin-top:-4px;}
</style>
</head>
<body>
<h1>Earth-Moon CR3BP &mdash; Minimum-Thrust Equilibrium ("Artificial Lagrange Point") Finder</h1>
<div class="wrap">
  <div class="left">
    <fieldset>
      <legend>Orbital Elements</legend>
      <div class="row"><label>Semi-major Axis (km)</label><input id="sma" value="326400"></div>
      <div class="row"><label>Eccentricity</label><input id="ecc" value="0.0"></div>
      <div class="row"><label>Inclination (deg)</label><input id="inc" value="5.0"></div>
      <div class="row"><label>RAAN (deg)</label><input id="raan" value="0.0"></div>
      <div class="row"><label>Arg. of Periapsis (deg)</label><input id="argp" value="0.0"></div>
      <div class="row"><label>True Anomaly (deg)</label><input id="nu" value="0.0"></div>
    </fieldset>
    <fieldset>
      <legend>Spacecraft / Optimizer</legend>
      <div class="row"><label>Mass (kg)</label><input id="mass" value="500"></div>
      <div class="row"><label>Max Continuous Thrust (N)</label><input id="maxThrust" value="50"></div>
      <div class="row"><label>Search Radius (km)</label><input id="searchRadius" value="50000"></div>
      <div class="row"><label>Optimization Tolerance</label><input id="tol" value="1e-8"></div>
      <div class="row"><label>Maximum Iterations</label><input id="maxIter" value="600"></div>
    </fieldset>
    <button id="runBtn">Run</button>
    <button id="plotBtn" class="secondary">Plot</button>
    <button id="exportBtn" class="secondary">Export CSV</button>
    <button id="validateBtn" class="secondary">Validate (L1&ndash;L5)</button>
    <div class="hint">Positions are in the Earth&ndash;Moon barycentric rotating frame.</div>
  </div>
  <div class="right">
    <fieldset>
      <legend>Output Panel</legend>
      <pre id="output">Enter inputs and click Run.</pre>
    </fieldset>
    <div id="plotDiv"></div>
    <div id="validateWrap"></div>
  </div>
</div>

<script>
/* ---------------- physics.js ---------------- */
const GM_EARTH = 398600.4418;
const GM_MOON = 4902.8000;
const D_EM = 384400.0;
const MU = GM_MOON / (GM_EARTH + GM_MOON);
const OMEGA = Math.sqrt((GM_EARTH + GM_MOON) / Math.pow(D_EM, 3));
const EARTH_POS = [-MU * D_EM, 0, 0];
const MOON_POS = [(1 - MU) * D_EM, 0, 0];
const EARTH_RADIUS = 6371.0;
const MOON_RADIUS = 1737.4;

function sub(a, b) { return [a[0]-b[0], a[1]-b[1], a[2]-b[2]]; }
function add(a, b) { return [a[0]+b[0], a[1]+b[1], a[2]+b[2]]; }
function norm(v) { return Math.sqrt(v[0]*v[0]+v[1]*v[1]+v[2]*v[2]); }

function gravityAccel(r, bodyPos, gm) {
  const d = sub(r, bodyPos);
  const dist = norm(d);
  const d3 = dist*dist*dist;
  return [-gm*d[0]/d3, -gm*d[1]/d3, -gm*d[2]/d3];
}

function naturalAccel(r) {
  const ae = gravityAccel(r, EARTH_POS, GM_EARTH);
  const am = gravityAccel(r, MOON_POS, GM_MOON);
  const ac = [OMEGA*OMEGA*r[0], OMEGA*OMEGA*r[1], 0];
  return [ae[0]+am[0]+ac[0], ae[1]+am[1]+ac[1], ae[2]+am[2]+ac[2]];
}

function distFromEarth(r) { return norm(sub(r, EARTH_POS)); }
function distFromMoon(r) { return norm(sub(r, MOON_POS)); }
```

```
/* ---------------- orbit.js ---------------- */
function coeToRV(a, e, incDeg, raanDeg, argpDeg, nuDeg, gm=GM_EARTH) {
  const inc=incDeg*Math.PI/180, raan=raanDeg*Math.PI/180, argp=argpDeg*Math.PI/180, nu=nuDeg*Math.PI/180;
  const p = a*(1-e*e);
  const rMag = p/(1+e*Math.cos(nu));
  const rPF = [rMag*Math.cos(nu), rMag*Math.sin(nu), 0];
  const vPF = [-Math.sqrt(gm/p)*Math.sin(nu), Math.sqrt(gm/p)*(e+Math.cos(nu)), 0];
  const cO=Math.cos(raan), sO=Math.sin(raan);
  const ci=Math.cos(inc), si=Math.sin(inc);
  const cw=Math.cos(argp), sw=Math.sin(argp);
  const R=[
    [cO*cw - sO*sw*ci, -cO*sw - sO*cw*ci, sO*si],
    [sO*cw + cO*sw*ci, -sO*sw + cO*cw*ci, -cO*si],
    [sw*si, cw*si, ci]
  ];
  const mv=(M,v)=>[M[0][0]*v[0]+M[0][1]*v[1]+M[0][2]*v[2], M[1][0]*v[0]+M[1][1]*v[1]+M[1][2]*v[2],
M[2][0]*v[0]+M[2][1]*v[1]+M[2][2]*v[2]];
  return { r: mv(R,rPF), v: mv(R,vPF) };
}

function orbitPath(a, e, incDeg, raanDeg, argpDeg, n=200, gm=GM_EARTH) {
  const pts = [];
  for (let i=0;i<n;i++) pts.push(coeToRV(a,e,incDeg,raanDeg,argpDeg, 360*i/(n-1), gm).r);
  return pts;
}

function eciToRotating(rEci, earthPosRotating) { return add(rEci, earthPosRotating); }

/* ---------------- optimizer.js (derivative-free Nelder-Mead, ball-constrained) ---------------- */
function projectToBall(r, r0, R) {
  const d = sub(r, r0);
  const dn = norm(d);
  if (dn <= R) return r;
  const s = R/dn;
  return [r0[0]+d[0]*s, r0[1]+d[1]*s, r0[2]+d[2]*s];
}

function nelderMead(f, x0, opts) {
  const n = x0.length;
  const alpha=1, gamma=2, rho=0.5, sigma=0.5;
  const maxIter = opts.maxIter || 500;
  const tol = opts.tol || 1e-14;
  const initStep = opts.initStep || 1;

  let simplex = [x0.slice()];
  for (let i=0;i<n;i++) { const x=x0.slice(); x[i]+=initStep; simplex.push(x); }
  let fvals = simplex.map(f);
  let iter = 0, converged = false;

  for (iter=1; iter<=maxIter; iter++) {
    const idx = fvals.map((v,i)=>i).sort((a,b)=>fvals[a]-fvals[b]);
    simplex = idx.map(i=>simplex[i]);
    fvals = idx.map(i=>fvals[i]);
    if (Math.abs(fvals[n]-fvals[0]) < tol) { converged = true; break; }

    const centroid = new Array(n).fill(0);
    for (let i=0;i<n;i++) for (let j=0;j<n;j++) centroid[j]+=simplex[i][j];
    for (let j=0;j<n;j++) centroid[j]/=n;

    const worst = simplex[n];
    const xr = centroid.map((c,j)=>c+alpha*(c-worst[j]));
    const fr = f(xr);

    if (fr < fvals[0]) {
      const xe = centroid.map((c,j)=>c+gamma*(xr[j]-c));
      const fe = f(xe);
      if (fe < fr) { simplex[n]=xe; fvals[n]=fe; } else { simplex[n]=xr; fvals[n]=fr; }
    } else if (fr < fvals[n-1]) {
      simplex[n]=xr; fvals[n]=fr;
    } else {
      const xc = centroid.map((c,j)=>c+rho*(worst[j]-c));
      const fc = f(xc);
      if (fc < fvals[n]) { simplex[n]=xc; fvals[n]=fc; }
      else { for (let i=1;i<=n;i++){ simplex[i]=simplex[0].map((v,j)=>v+sigma*(simplex[i][j]-v));
fvals[i]=f(simplex[i]); } }
    }
  }
  const idx = fvals.map((v,i)=>i).sort((a,b)=>fvals[a]-fvals[b]);
  return { x: simplex[idx[0]], f: fvals[idx[0]], iterations: iter, converged };
}
```

```
function findMinThrustEquilibrium(r0, mass, R, tol, maxIter, maxThrust) {
  const objective = (r) => { const a = naturalAccel(projectToBall(r, r0, R)); return
a[0]*a[0]+a[1]*a[1]+a[2]*a[2]; };
  const initStep = Math.max(R*0.1, 10);
  const res = nelderMead(objective, r0.slice(), { maxIter, tol: tol*tol, initStep });
  const rSol = projectToBall(res.x, r0, R);
  const aNat = naturalAccel(rSol);
  const T = aNat.map(a => -1000*mass*a);
  const Tmag = norm(T);
  const residual = norm(aNat.map((a,i) => a + (T[i]/mass)*1e-3));
  return {
    position: rSol, thrust: T, thrustMagnitude: Tmag,
    distanceEarth: distFromEarth(rSol), distanceMoon: distFromMoon(rSol),
    success: res.converged, iterations: res.iterations, residualMagnitude: residual,
    withinThrustLimit: Tmag <= maxThrust,
  };
}

/* ---------------- validation.js ---------------- */
function bisectRoot(f, lo, hi, n=20000) {
  let prevX=lo, prevF=f(lo);
  for (let i=1;i<=n;i++) {
    const x = lo + (hi-lo)*i/n;
    const fx = f(x);
    if (isFinite(prevF) && isFinite(fx) && prevF*fx < 0) {
      let a=prevX,b=x,fa=prevF;
      for (let k=0;k<80;k++) { const m=(a+b)/2, fm=f(m); if (fa*fm<=0) b=m; else { a=m; fa=fm; } }
      return (a+b)/2;
    }
    prevX=x; prevF=fx;
  }
  return null;
}

function knownLagrangePoints() {
  const ax = (x) => naturalAccel([x,0,0])[0];
  const xE=EARTH_POS[0], xM=MOON_POS[0], eps=1.0;
  const xL1 = bisectRoot(ax, xE+eps, xM-eps);
  const xL2 = bisectRoot(ax, xM+eps, xM+2*D_EM);
  const xL3 = bisectRoot(ax, xE-2*D_EM, xE-eps);
  return {
    L1:[xL1,0,0], L2:[xL2,0,0], L3:[xL3,0,0],
    L4:[D_EM*(0.5-MU), D_EM*Math.sqrt(3)/2, 0],
    L5:[D_EM*(0.5-MU), -D_EM*Math.sqrt(3)/2, 0],
  };
}

function validateAll(mass=1000.0, searchRadius=5000.0, tol=1e-10, maxIter=400) {
  const known = knownLagrangePoints();
  const out = {};
  for (const name of ["L1","L2","L3","L4","L5"]) {
    const pos = known[name];
    const res = findMinThrustEquilibrium(pos, mass, searchRadius, tol, maxIter, 1e12);
    out[name] = { known: pos, computed: res.position, posError: norm(sub(res.position,pos)), residual:
res.residualMagnitude, success: res.success };
  }
  return out;
}

/* ---------------- plots.js (Plotly) ---------------- */
function sphereMesh(center, radius, n=24) {
  const x=[], y=[], z=[];
  for (let i=0;i<=n;i++) {
    const u = Math.PI*i/n;
    const xr=[], yr=[], zr=[];
    for (let j=0;j<=n;j++) {
      const v = 2*Math.PI*j/n;
      xr.push(center[0]+radius*Math.sin(u)*Math.cos(v));
      yr.push(center[1]+radius*Math.sin(u)*Math.sin(v));
      zr.push(center[2]+radius*Math.cos(u));
    }
    x.push(xr); y.push(yr); z.push(zr);
  }
  return {x,y,z};
}

function plotSystem(orbitPts, eqPos, thrustVec) {
  const earthSph = sphereMesh(EARTH_POS, EARTH_RADIUS);
  const moonSph = sphereMesh(MOON_POS, MOON_RADIUS);
```

```
  const traceEarth = { type:'surface', x:earthSph.x, y:earthSph.y, z:earthSph.z, showscale:false,
colorscale:[[0,'royalblue'],[1,'royalblue']], name:'Earth' };
  const traceMoon = { type:'surface', x:moonSph.x, y:moonSph.y, z:moonSph.z, showscale:false,
colorscale:[[0,'dimgray'],[1,'dimgray']], name:'Moon' };
  const traceOrbit = { type:'scatter3d', mode:'lines', x:orbitPts.map(p=>p[0]), y:orbitPts.map(p=>p[1]),
z:orbitPts.map(p=>p[2]), line:{color:'darkorange', width:3}, name:'Input Orbit' };
  const traceEq = { type:'scatter3d', mode:'markers', x:[eqPos[0]], y:[eqPos[1]], z:[eqPos[2]],
marker:{color:'red', size:6}, name:'Equilibrium Point' };

  const tmag = norm(thrustVec);
  const traces = [traceEarth, traceMoon, traceOrbit, traceEq];
  if (tmag > 0) {
    const scale = (D_EM*0.15)/tmag;
    const tip = [eqPos[0]+thrustVec[0]*scale, eqPos[1]+thrustVec[1]*scale, eqPos[2]+thrustVec[2]*scale];
    traces.push({ type:'scatter3d', mode:'lines+markers', x:[eqPos[0],tip[0]], y:[eqPos[1],tip[1]],
z:[eqPos[2],tip[2]],
      line:{color:'limegreen', width:6}, marker:{size:[0,4], color:'limegreen'}, name:'Thrust Vector' });
  }

  const layout = {
    paper_bgcolor:'#111214', plot_bgcolor:'#111214', font:{color:'#e6e6e6'},
    scene:{ xaxis:{title:'x [km]', gridcolor:'#333'}, yaxis:{title:'y [km]', gridcolor:'#333'}, zaxis:{title:'z
[km]', gridcolor:'#333'},
      bgcolor:'#111214', aspectmode:'data' },
    margin:{l:0,r:0,t:20,b:0}, legend:{x:0, y:1},
  };
  Plotly.newPlot('plotDiv', traces, layout, {responsive:true});
}

/* ---------------- gui.js ---------------- */
function readInputs() {
  const g = id => document.getElementById(id).value;
  const inp = {
    sma: parseFloat(g('sma')), ecc: parseFloat(g('ecc')), inc: parseFloat(g('inc')),
    raan: parseFloat(g('raan')), argp: parseFloat(g('argp')), nu: parseFloat(g('nu')),
    mass: parseFloat(g('mass')), maxThrust: parseFloat(g('maxThrust')),
    searchRadius: parseFloat(g('searchRadius')), tol: parseFloat(g('tol')), maxIter: parseInt(g('maxIter')),
  };
  for (const [k,v] of Object.entries(inp)) if (!isFinite(v)) throw new Error('Invalid numeric input: '+k);
  if (inp.sma<=0) throw new Error('Semi-major axis must be positive.');
  if (inp.ecc<0 || inp.ecc>=1) throw new Error('Eccentricity must be in [0,1).');
  if (inp.mass<=0) throw new Error('Mass must be positive.');
  if (inp.searchRadius<=0) throw new Error('Search radius must be positive.');
  if (inp.maxIter<=0) throw new Error('Maximum iterations must be positive.');
  return inp;
}

let lastRun = null;

function fmt(n, d=6){ return Number(n).toFixed(d); }

function onRun() {
  let inp;
  try { inp = readInputs(); } catch(e) { alert(e.message); return; }

  const rv = coeToRV(inp.sma, inp.ecc, inp.inc, inp.raan, inp.argp, inp.nu);
  const r0 = eciToRotating(rv.r, EARTH_POS);
  const result = findMinThrustEquilibrium(r0, inp.mass, inp.searchRadius, inp.tol, inp.maxIter, inp.maxThrust);

  lastRun = { inputs: inp, r0, result };

  const p = result.position, t = result.thrust;
  const lines = [
    'EQUILIBRIUM RESULT',
    '='.repeat(60),
    `Initial orbit position (rotating frame): [${fmt(r0[0],3)}, ${fmt(r0[1],3)}, ${fmt(r0[2],3)}] km`,
    '',
    'Equilibrium Position (rotating frame):',
    `  x = ${fmt(p[0])} km`, `  y = ${fmt(p[1])} km`, `  z = ${fmt(p[2])} km`,
    '',
    'Required Thrust:',
    `  Tx = ${fmt(t[0])} N`, `  Ty = ${fmt(t[1])} N`, `  Tz = ${fmt(t[2])} N`,
    `  Thrust Magnitude = ${fmt(result.thrustMagnitude)} N`,
    '',
    `Distance from Earth = ${fmt(result.distanceEarth,3)} km`,
    `Distance from Moon  = ${fmt(result.distanceMoon,3)} km`,
    '',
    `Optimization Success    = ${result.success}`,
```

```
    `Within Thrust Limit      = ${result.withinThrustLimit}`,
    `Iterations               = ${result.iterations}`,
    `Residual Force Magnitude = ${result.residualMagnitude.toExponential(3)}`,
  ];
  document.getElementById('output').textContent = lines.join('\n');
}

function onPlot() {
  if (!lastRun) { alert('Run the optimizer first.'); return; }
  const inp = lastRun.inputs;
  const orbitEci = orbitPath(inp.sma, inp.ecc, inp.inc, inp.raan, inp.argp);
  const orbitRot = orbitEci.map(p => eciToRotating(p, EARTH_POS));
  plotSystem(orbitRot, lastRun.result.position, lastRun.result.thrust);
}

function onExportCsv() {
  if (!lastRun) { alert('Run the optimizer first.'); return; }
  const inp = lastRun.inputs, r = lastRun.result;
  const rows = [['Section','Field','Value']];
  rows.push(['Input','Semi-major Axis (km)', inp.sma]);
  rows.push(['Input','Eccentricity', inp.ecc]);
  rows.push(['Input','Inclination (deg)', inp.inc]);
  rows.push(['Input','RAAN (deg)', inp.raan]);
  rows.push(['Input','Argument of Periapsis (deg)', inp.argp]);
  rows.push(['Input','True Anomaly (deg)', inp.nu]);
  rows.push(['Input','Mass (kg)', inp.mass]);
  rows.push(['Input','Maximum Continuous Thrust (N)', inp.maxThrust]);
  rows.push(['Input','Search Radius (km)', inp.searchRadius]);
  rows.push(['Input','Optimization Tolerance', inp.tol]);
  rows.push(['Input','Maximum Iterations', inp.maxIter]);
  rows.push(['Output','Equilibrium x (km)', r.position[0]]);
  rows.push(['Output','Equilibrium y (km)', r.position[1]]);
  rows.push(['Output','Equilibrium z (km)', r.position[2]]);
  rows.push(['Output','Thrust Tx (N)', r.thrust[0]]);
  rows.push(['Output','Thrust Ty (N)', r.thrust[1]]);
  rows.push(['Output','Thrust Tz (N)', r.thrust[2]]);
  rows.push(['Output','Thrust Magnitude (N)', r.thrustMagnitude]);
  rows.push(['Output','Distance from Earth (km)', r.distanceEarth]);
  rows.push(['Output','Distance from Moon (km)', r.distanceMoon]);
  rows.push(['Output','Optimization Success', r.success]);
  rows.push(['Output','Within Thrust Limit', r.withinThrustLimit]);
  rows.push(['Output','Iterations', r.iterations]);
  rows.push(['Residual Error','Residual Force Magnitude', r.residualMagnitude]);

  const csv = rows.map(row => row.join(',')).join('\n');
  const blob = new Blob([csv], {type:'text/csv'});
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url; a.download = 'cr3bp_equilibrium_results.csv';
  document.body.appendChild(a); a.click(); document.body.removeChild(a);
  URL.revokeObjectURL(url);
}

function onValidate() {
  let mass=1000, tol=1e-10, maxIter=400;
  try { const inp = readInputs(); mass=inp.mass; tol=inp.tol; maxIter=Math.max(inp.maxIter,300); } catch(e) {}

  const results = validateAll(mass, 5000.0, tol, maxIter);
  let html = '<fieldset><legend>Validation &mdash; Known Lagrange Points</legend><table>';
  html += '<tr><th>Point</th><th>Known Position (km)</th><th>Computed Position (km)</th><th>Position Error
(km)</th><th>Residual Error</th><th>Success</th></tr>';
  for (const name of ['L1','L2','L3','L4','L5']) {
    const r = results[name];
    const k = r.known.map(v=>v.toFixed(1)).join(', ');
    const c = r.computed.map(v=>v.toFixed(1)).join(', ');
    html +=
`<tr><td>${name}</td><td>[${k}]</td><td>[${c}]</td><td>${r.posError.toExponential(3)}</td><td>${r.residual.toEx
ponential(3)}</td><td>${r.success}</td></tr>`;
  }
  html += '</table></fieldset>';
  document.getElementById('validateWrap').innerHTML = html;
}

document.getElementById('runBtn').addEventListener('click', onRun);
document.getElementById('plotBtn').addEventListener('click', onPlot);
document.getElementById('exportBtn').addEventListener('click', onExportCsv);
document.getElementById('validateBtn').addEventListener('click', onValidate);
</script>
</body>
</html>
```